

ENGLISH FOR WORK / SEPTEMBER 2026

Software Development English



Teacher's guide

Teach practical workplace communication with specific cases, clear language targets, and a repeatable feedback process.

PRACTICE THAT TRANSFERS TO WORK

Read a realistic case. Study complete conversations. Find the right language, speak, get feedback, and try again.

Eight lessons | Intermediate to advanced | Independent or classroom study

For Software developers, application engineers, code reviewers, and maintainers.

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

All scenarios, figures, and model responses are fictional teaching material. Use invented details in practice.

[Open this course on English Ladder](#)

Your course at a glance

Use the lesson numbers to match this document with the website and the other guides.

01 Clarifying interface contracts

Resolve an interface ambiguity using concrete input and outcome pairs.

02 Debugging from reproducible evidence

Explain a debugging hypothesis and propose evidence that distinguishes it.

03 Giving actionable code-review feedback

Give precise review feedback that separates an acceptable edit from an unintended behavior change.

04 Requesting dependency evidence

Ask for evidence that makes a dependency upgrade reviewable.

05 Explaining what a test actually exercises

Explain what a test verifies and what its substitutes leave outside scope.

06 Comparing performance tradeoffs fairly

Describe a measured benefit and its cost without generalizing beyond the benchmark.

07 Discussing retries and shared state

Explain why repeating a state-changing request needs an explicit behavior contract.

08 Handing over a release with explicit limits

Give a release handover that distinguishes installed software from enabled behavior and verified recovery.

Choose your pace

Quick practice: spend 15 minutes reading one conversation and rehearsing one scenario. Full lesson: allow 45-60 minutes for language work, three conversations, two speaking scenarios, and feedback.

Level support

B1 learners can use the frames and a partner. B2 learners can try a response before reading the model. C1 learners can add the harder follow-up and reduce preparation time. These are teaching suggestions, not certified CEFR level ratings.

After practicing: open the AI prompt workshop to choose a vocabulary, grammar, role-play, or new-dialogue activity.

A 60-minute lesson that works

Prepare the case and the learner pages. Keep the model response hidden until learners have attempted their own. Choose two field terms that are important to understanding the case.

0-5 minutes | Activate experience

Ask learners to name a comparable situation without sharing confidential details. Introduce the communication goal.

5-12 minutes | Read and sort facts

Read the case. Learners identify confirmed facts, unknowns, the audience, and the immediate communication need.

12-22 minutes | Notice and practice language

Explain the workshop pattern. Work through the vocabulary and editing example, then discuss both language checks and the reasons behind the choices.

22-35 minutes | 05 • Conversations

Read one ten-turn conversation aloud, notice a useful expression, then compare the other two. Identify each speaker's purpose and how the exchange ends.

35-47 minutes | 06 • Say it

Rehearse both speaking scenarios from the workbook. Give two minutes of preparation, switch roles, and use the success checks. Add the complication only after a successful first round.

47-55 minutes | Compare and revise

Reveal the model. Discuss alternative acceptable wording. Give one meaning correction and one language correction, then repeat.

55-60 minutes | Retrieve and transfer

Close the materials. Learners recall two expressions and identify one communication habit to use in another situation.

Adapt the session

For 45 minutes, study one complete conversation and rehearse one scenario; assign the others for oral follow-up. For 90 minutes, rehearse all three conversations and give both scenarios a second round. More time should produce more meaningful practice, not longer explanations.

LESSON 01 / FACILITATOR NOTES

Clarifying interface contracts

Resolve an interface ambiguity using concrete input and outcome pairs.

CASE

A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

Teach this move

Explain the specified cases before asking about the gap. 'Optional' concerns whether a field must be present; 'nullable' concerns whether null is accepted. Neither automatically tells a caller what an empty string does.

Vocabulary to check: interface contract, nullable field, validation error, backward compatibility.

Model response

The contract already distinguishes an omitted nickname from an empty string: one leaves the value unchanged and the other clears it. Could we agree what null means before the client starts sending it? At present, null is unspecified, so I cannot describe it as another supported way to clear the field.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The revision distinguishes three inputs instead of grouping them under the ambiguous word blank.

Second-round challenge

A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

Language-check answers

1. B - unspecified Correct. The meaning of null is not defined.
2. A - whereas Correct. Whereas contrasts different properties.

05 • Conversations

1. What does null do to a nickname? - Two fictional developers resolve an unspecified input in the profile contract.

2. An empty list is a successful result - Fictional developers clarify the difference between no search matches and a failed search.

3. A renamed response field breaks an older caller - A fictional code review identifies a compatibility concern in a response change.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | A required currency without a default

A fictional pricing interface requires a currency code. A client omits it, assuming dollars. The contract supplies no default. The interface owner can confirm that omission is invalid; no conversion is involved.

Role A: Client developer: ask whether omission has a supported meaning and restate the answer.

Role B: Interface owner: explain that currency is required and a value must be supplied explicitly.

Possible opening: Does an omitted currency mean dollars in this contract, or is it invalid?

Success checks

- Separate example data from the contract.
- Confirm currency must be explicit.
- Avoid inventing a conversion or default behavior.

Add a complication: Someone points to a dollar sign in an example and calls it a default rule.

LESSON 02 / FACILITATOR NOTES

Debugging from reproducible evidence

Explain a debugging hypothesis and propose evidence that distinguishes it.

CASE

A fictional import fails on a three-row file containing an empty date field. A two-row file without that row succeeds. The stack trace points to date parsing, but no controlled test has isolated the empty field as the cause.

Teach this move

A stack trace shows a path associated with failure, not necessarily the original cause. When two examples differ in several ways, identify the confounding differences before claiming that one field explains the failure.

Vocabulary to check: reproduction, stack trace, hypothesis, minimal example.

Model response

The failing three-row file contains an empty date, and the stack trace reaches date parsing. Those observations make the empty field a useful hypothesis, not yet a demonstrated cause. Could we compare otherwise identical examples with and without that value? The smaller passing file also changed the row count, so it does not isolate the field.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The revised claim matches the evidence and names the untested explanation.

Second-round challenge

The same failing file succeeds once after a retry. Explain why one success does not establish a fix.

Language-check answers

1. A - row count Correct. Both the date content and row count differ.
2. D - suggests Correct. Suggests presents a supported but unconfirmed explanation.

05 • Conversations

1. The empty date is still a hypothesis - Fictional developers discuss a failing three-row import and a passing two-row file.

2. The error line is not necessarily where the defect began - Fictional developers inspect a trace from a report formatter.

3. One successful rerun doesn't establish a correction - Fictional developers discuss intermittent behavior without claiming a fix.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | A failure that depends on input order

In a fictional example, records A then B produce a duplicate label, while B then A do not. The same records and build are used. No internal cause has been established.

Role A: Developer: describe the observed ordering difference and propose a controlled comparison.

Role B: Maintainer: ask what is identical and whether the result proves a particular internal defect.

Possible opening: The same two records produce different results when their order changes.

Success checks

- Preserve the order-sensitive observation.
- Keep the internal explanation unconfirmed.
- Agree a smaller order comparison with recorded outputs.

Add a complication: A colleague declares the data corrupted even though the records are unchanged.

LESSON 03 / FACILITATOR NOTES

Giving actionable code-review feedback

Give precise review feedback that separates an acceptable edit from an unintended behavior change.

CASE

A fictional pull request is titled 'rename export helper.' Its diff also moves a notification call outside a success branch, so failed exports would now notify the user as if complete. The naming change is acceptable; the behavior change is unintended.

Teach this move

Point to the observed change, explain the resulting behavior, and request a bounded revision. Refactoring normally preserves observable behavior; the label on a pull request does not guarantee that its diff does so.

Vocabulary to check: pull request, diff, refactoring, side effect.

Model response

The helper rename is clear. In the same diff, the notification call moves outside the success branch, which would also notify after a failed export. Could you keep that call conditional on success and add a check for the failure path? My concern is the changed side effect, not your choice of helper name.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The review now names the defect and a concrete revision rather than judging the author or the entire change.

Second-round challenge

A second reviewer adds an optional naming preference. Label it optional while keeping the failure-path correction clearly required.

Language-check answers

1. D - side effect Correct. The notification is an observable effect outside the helper's result.
2. C - preserve Correct. Preserve asks to retain the existing condition.

05 · Conversations

1. A rename that also moves a notification - A fictional author and reviewer resolve an unintended side effect in a pull request.
2. An optional suggestion among required changes - Fictional reviewers distinguish a readability preference from an incorrect boundary check.
3. A helper extracts more behavior than expected - Fictional developers discuss a refactor that unexpectedly reorders an observable event.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | A review comment that needs an example

A fictional review says 'This loses precision.' The author rounded a displayed duration, but the underlying calculation remains unchanged. The reviewer is concerned only about the displayed boundary between 0.9 and 1.0 seconds.

Role A: Author: ask which value, output, and consequence the comment refers to.

Role B: Reviewer: give the 0.9-to-1.0 display example and distinguish it from calculation accuracy.

Possible opening: Could you identify which output loses precision and whether your concern is display or calculation?

Success checks

- Ask for the concrete example.
- Separate presentation from underlying computation.
- Agree a focused review of display rounding.

Add a complication: A teammate proposes rewriting the calculation before the display concern is clarified.

LESSON 04 / FACILITATOR NOTES

Requesting dependency evidence

Ask for evidence that makes a dependency upgrade reviewable.

CASE

A fictional application directly uses package ParcelKit, which brings in TextCore indirectly. A proposed ParcelKit upgrade changes the resolved TextCore version. The change note mentions a deprecated parsing entry point; the team's usage and replacement compatibility are unknown.

Teach this move

Direct and transitive describe how a dependency enters the application. Deprecation warns that an interface should be replaced or may be removed; it does not by itself establish removal now. Tool and package behavior must be checked in their actual documentation.

Vocabulary to check: direct dependency, transitive dependency, lockfile, deprecation.

Model response

Could you identify whether our application or ParcelKit calls the deprecated parsing entry point, and link the relevant upgrade note? The proposed change also alters the resolved TextCore version, so please show the dependency path and lockfile difference. We need that evidence before calling this a drop-in update with no caller changes.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The revision requests evidence rather than treating a version label as a compatibility guarantee.

Second-round challenge

The package name is unchanged. Explain why that alone does not establish identical behavior across versions.

Language-check answers

1. C - transitive Correct. The application receives it through another dependency.
2. B - removal Correct. An interface may be deprecated before removal.

05 • Conversations

1. The package we didn't list directly - Fictional developers identify evidence needed for a ParcelKit upgrade review.

2. A lockfile difference without a source change - Fictional maintainers discuss why two builds resolved different dependency sets.

3. Deprecated is not removed - A fictional maintainer clarifies an upgrade note with a caller's developer.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | Two components request different versions

In a fictional dependency report, App requests FormatKit 1.4; App -> ViewPlug requests FormatKit 2.1. The supplied graph retains both versions at those paths; the tool's resolution rules remain unreviewed.

Role A: Developer: ask what was actually resolved and where each request originates.

Role B: Maintainer: show the two requests and state that the tool's resolution rules need checking.

Possible opening: These are two requested versions; which versions actually appear in the resolved graph?

Success checks

- Distinguish requested from resolved versions.
- Identify both dependency paths.
- Avoid claiming a universal resolution rule.

Add a complication: Someone assumes the higher version always wins in every package tool.

LESSON 05 / FACILITATOR NOTES

Explaining what a test actually exercises

Explain what a test verifies and what its substitutes leave outside scope.

CASE

A fictional save function's unit test replaces storage with a test double that always reports success. It checks the success message but never exercises a storage rejection. A previous bug incorrectly showed success after rejection; the real storage adapter is not tested here.

Teach this move

Name the real component, the replaced collaborator, and the assertion. A test can run quickly and still omit the behavior in question. A passing result supports its actual checks, not every possible interaction.

Vocabulary to check: unit test, integration test, test double, regression test.

Model response

This unit test checks the save function's success path using a controlled substitute for storage. Because that substitute always succeeds, it does not exercise the rejection that triggered the earlier bug. A regression case should check the visible result when rejection is simulated. An integration test of the real adapter answers a different question.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The revision identifies the checked behavior and the boundaries of the evidence.

Second-round challenge

A teammate proposes checking only that no exception was thrown. Explain why the visible message needs its own assertion.

Language-check answers

1. B - always reports success Correct. An always-successful substitute never reaches the disputed path.
2. A - regression Correct. It guards against reintroducing the earlier defect.

05 · Conversations

1. The storage substitute never rejects - Fictional developers examine why a unit test missed a misleading success message.
2. A test that calls code but checks no result - Fictional developers strengthen the stated purpose of a parsing test.
3. A test double with the wrong contract - Fictional developers find a mismatch between a controlled substitute and a real collaborator.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | A test name overstates its assertion

A fictional test named 'saves and reloads settings' checks only the save response. It uses a test double and never performs a reload. The author can rename it and propose a distinct persistence check.

Role A: Reviewer: explain the mismatch between the test name and observed actions.

Role B: Author: distinguish a response check from a read-after-save interaction.

Possible opening: This test checks the save response, but where does it reload the settings?

Success checks

- State exactly what is asserted.
- Avoid treating a response as proof of persistence.
- Agree a truthful name and a separate interaction question.

Add a complication: A teammate says the positive response makes the reload check redundant.

LESSON 06 / FACILITATOR NOTES

Comparing performance tradeoffs fairly

Describe a measured benefit and its cost without generalizing beyond the benchmark.

CASE

In a fictional controlled benchmark on the same input and machine, implementation A has median latency 40 milliseconds and memory use 80 megabytes; B has 25 milliseconds and 160 megabytes. The available memory budget is 120 megabytes. Tail latency and production workloads are unmeasured.

Teach this move

Latency measures delay; throughput measures work completed over time. A median is not a maximum, and a result for one workload is not a universal prediction. Use the same units and the stated resource constraint.

Vocabulary to check: latency, throughput, benchmark, memory footprint.

Model response

B has lower median latency in this benchmark, but it uses one hundred sixty megabytes against our one-hundred-twenty-megabyte budget. A is slower at the median and stays within that budget. I would not call B the better fit without addressing memory. Neither result establishes tail latency or performance on unmeasured production workloads.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The revised comparison includes the limiting resource and the boundary of the measurements.

Second-round challenge

The faster result came from a warmed cache in a later rerun. Ask for matched conditions before comparing that rerun with a cold run.

Language-check answers

1. A - Implementation A Correct. A's 80 megabytes is below 120.
2. D - maximum Correct. The maximum identifies the greatest observed value in the sample.

05 · Conversations

1. Faster at the median, outside the memory budget - Fictional developers compare implementations A and B under the supplied benchmark.
2. Throughput rises while individual waits grow - Fictional developers interpret two different measures in a batching experiment.
3. A warm run beside a cold run - Fictional developers correct an unfair benchmark comparison.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | The average hides the slow end

A fictional benchmark reports unchanged average latency, but its slowest measured group takes longer after a change. Sample size is forty runs; production representativeness is unknown.

Role A: Developer: explain how an unchanged average can coexist with slower individual observations.

Role B: Reviewer: ask for the distribution and comparable conditions, without assuming every user is slower.

Possible opening: The average is unchanged, but the slow end of this sample has moved.

Success checks

- Distinguish the average from the distribution.
- Keep the forty-run scope visible.
- Avoid claiming a production-wide effect.

Add a complication: A stakeholder wants to say there is no performance change because the average matches.

LESSON 07 / FACILITATOR NOTES

Discussing retries and shared state

Explain why repeating a state-changing request needs an explicit behavior contract.

CASE

A fictional reservation client times out before receiving a reply. The reservation may already have been created. A colleague proposes automatically submitting a second request as a new reservation. The service's duplicate-request behavior is unspecified; no production change is authorized.

Teach this move

A client may fail to receive a reply even when a service completed work. Idempotency concerns the intended effect of repetition, not a promise that every response or timing is identical. Discuss the fictional contract rather than prescribing a production retry mechanism.

Vocabulary to check: race condition, idempotency, transaction, invariant.

Model response

I would not treat the timeout as proof that no reservation was created. A second request submitted as new could create another reservation if the first one completed. Could we define how repeated requests are recognized and what result the client should receive? We need that contract before recommending an automatic retry for this operation.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The revision separates the client's observation from the service's unconfirmed state.

Second-round challenge

Someone says wrapping work in a transaction automatically settles all duplicate-request behavior. Ask which invariant and concurrency guarantees actually apply.

Language-check answers

1. D - unknown Correct. A missing reply does not reveal whether creation completed.
2. C - Idempotency Correct. Idempotency limits the intended effect of repetition.

05 · Conversations

1. The reply timed out, not necessarily the reservation - Fictional developers discuss a proposed automatic repeat of a state-changing request.
2. Two callers each see the last slot - Fictional developers discuss an overlapping-operation example without choosing a database technique.
3. A repeated command is not a new intention - Fictional developers clarify a request identifier's meaning in a proposed contract.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | Two editors save different versions

In a fictional editor, two people open revision 6. One saves revision 7. The other's later save still refers to revision 6. Conflict handling is unspecified; neither edit is known to be dispensable.

Role A: Developer: describe the competing updates and ask what the interface should communicate.

Role B: Reviewer: insist that the intended conflict outcome be defined before choosing a mechanism.

Possible opening: The second save is based on an older revision; what outcome should the caller expect?

Success checks

- Identify the older base revision.
- Avoid inventing a universal last-writer rule.
- State the unresolved contract question about competing edits.

Add a complication: A colleague assumes the later request must always overwrite the earlier one.

LESSON 08 / FACILITATOR NOTES

Handing over a release with explicit limits

Give a release handover that distinguishes installed software from enabled behavior and verified recovery.

CASE

Fictional build R18 is deployed to staging. Its new search view is disabled by a feature flag. A schema migration adds an optional field, but compatibility with the previous application build is untested. Lee takes the release review; production rollout is undecided.

Teach this move

A deployed artifact may contain disabled behavior. Rolling back an application does not automatically undo data changes. State which environment and build were checked and name the remaining review without promising recovery.

Vocabulary to check: build artifact, feature flag, rollback, schema migration.

Model response

R18 is deployed to staging, with the new search view disabled by its feature flag. The optional-field migration is present there, but compatibility with the previous application build remains untested. Lee, please take that compatibility check into the release review. No production rollout or successful rollback has been confirmed by these staging facts.

Listen for, then coach

Ask learners to point to the case fact behind each important statement. Accept different wording when it preserves meaning, fits the listener, and does not invent an outcome. The revision avoids conflating staging deployment, feature exposure, and verified rollback.

Second-round challenge

A manager calls the disabled feature fully tested because the build started. Explain which user path has not been exercised.

Language-check answers

1. C - disabled Correct. The case states that exposure is disabled.
2. B - separate concerns Correct. Reverting application code does not itself establish a data reversal.

05 · Conversations

1. R18 is installed but search is disabled - A fictional developer hands the release review to Lee.
2. The artifact differs from the one tested - Fictional developers catch a release handover mismatch.
3. Switching a flag off doesn't erase prior changes - Fictional developers clarify the scope of disabling a feature after a trial.

Use the complete ten-turn scripts in the learner workbook and conversation lab. Read with roles, identify the decision or open question, then replay without reading.

06 · Say it: second scenario | A rollout pauses after a bounded trial

A fictional feature is enabled for ten internal test accounts. Two report missing labels. The release lead pauses further exposure; existing evidence covers only those ten accounts and does not establish an external impact.

Role A: Developer: report the observed failures, exposure boundary, and paused expansion.

Role B: Release reviewer: ask what is known about the other eight accounts and external users.

Possible opening: Two of ten internal test accounts reported missing labels, and further exposure is paused.

Success checks

- Preserve the ten-account scope.
- Distinguish reports from complete checks of the other accounts.
- Name the remaining investigation without expanding the impact claim.

Add a complication: A stakeholder asks to describe every user as affected.

Give feedback that helps

Score each criterion 0, 1, or 2. This is a practice rubric, not a professional qualification or CEFR test. Do not award or remove points for a particular accent.

Meaning and accuracy

Keeps the case facts accurate; clearly separates confirmed and unknown information.

Clarity and language

Uses understandable sentences and explains specialist terms when the listener needs it.

Interaction and tone

Responds to the other person's question, checks understanding, and uses an appropriate tone.

Task completion

Makes the requested action or unresolved question clear without inventing authority or facts.

Use the same scale for each criterion

0 - Not yet: The reader or listener cannot yet recover the intended meaning or action.

1 - With support: The message works after a prompt, clarification, or revision.

2 - Independently: The message is clear and accurate without a prompt.

Feedback sequence

First, identify a successful phrase. Next, identify one point where meaning or accuracy needs work. Offer one useful language correction, allow a repeat, and compare the two attempts. A total out of eight describes this performance only; do not translate it into a proficiency level.

Final challenge

Choose a case not rehearsed today. The learner gives a one-minute response, answers two follow-up questions, and switches roles and repeats the exchange. Add the lesson's second-round challenge. Compare the result with an earlier attempt using the four criteria.

Use the language responsibly

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Meaning before performance

Use specialist terms only when they help the listener. Explain unfamiliar words, preserve uncertainty, and ask when an instruction or responsibility is unclear. The model responses illustrate language; they do not authorize professional actions.

Sources and further reading

The cases, workshops, and explanations are original teaching material. These references provide language frameworks and selected professional context. References were checked in September 2026; consult current local guidance for actual work.

Digital.gov: plain-language guidance

<https://digital.gov/guides/plain-language>

Council of Europe: CEFR descriptors

<https://www.coe.int/en/web/common-european-framework-reference-languages/cefr-descriptors>

U.S. Bureau of Labor Statistics: Software Developers, Quality Assurance Analysts, and Testers

<https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>

NIST SP 800-218: Secure Software Development Framework, Version 1.1

<https://csrc.nist.gov/pubs/sp/800/218/final>

Your next step

Choose one expression you can use in a genuine work situation. Rehearse it with fictional details, then adapt the wording to your role and audience.